

datalab 报告

姓名：赵修齐 学号：2025201723

总分	bitAnd	bitXor	samesign	logtwo	byteSwap	reverse	logicalShift	leftBitCount	float_i2f	floatScale2	float64_f2i	floatPower2
37.00	1.00	1.00	2.00	4.00	4.00	3.00	3.00	4.00	4.00	4.00	3.00	4.00

test 截图：

```
● zxq38776277@zxq18638776277:~/projects/datalab2026$ make
python3 test.py
gcc -O0 -Wall -std=gnu99 -lm -o btest bits.c btest.c decl.c tests.c
rm -f *.o btest fshow ishow *~ yacctab.py lextab.py result.txt .autograder_result
gcc -O0 -Wall -std=gnu99 -lm -o btest bits.c btest.c decl.c tests.c
gcc -O0 -Wall -std=gnu99 -o fshow fshow.c
gcc -O0 -Wall -std=gnu99 -o ishow ishow.c
Make success.
bitAnd          1/1:      PASS
bitXor          1/1:      PASS
samesign        2/2:      PASS
logtwo          4/4:      PASS
byteSwap        4/4:      PASS
reverse         3/3:      PASS
logicalShift    3/3:      PASS
leftBitCount    4/4:      PASS
float_i2f       4/4:      PASS
floatScale2     4/4:      PASS
float64_f2i     3/3:      PASS
floatPower2     4/4:      PASS
Total points: 37
```

解题报告

亮点

1. logtwo: 采用分段二分定位最高有效位, 运算次数少, 避免逐位循环。
2. leftBitCount: 先对原数取反, 再通过 16、8、4、2、1 位分组统计前导零。

logtwo

关键代码如下:

```
result = (v > 0xFFFF) << 4;  
v = v >> result;  
shift = (v > 0xFF) << 3;  
result = result | shift;  
v = v >> shift;  
/* 按相同方法继续检查 4 位和 2 位 */  
result = result | (v >> 1);
```

对于正整数, logtwo(v) 等于其二进制最高有效位的编号。程序先判断最高位是否位于高 16 位, 再依次把判断范围缩小到 8、4、2、1 位。比较表达式产生 0 或 1, 左移后分别得到 16、8、4、2; 确定一个区间后, 将 v 右移相应位数, 继续检查剩余部分。

各次得到的位移量占据不同的二进制位, 因此可使用按位或合并。例如 16 与 8 按位或得到 24, 与相加结果相同。该方法在固定次数内完成计算, 不需要逐位循环。

leftBitCount

关键代码如下:

```
int n = ~x;  
int r1 = (!(n >> 16)) << 4;  
n = n << r1;  
int r2 = (!(n >> 24)) << 3;  
n = n << r2;  
/* 按相同方法继续检查 4、2、1 位 */  
return r1 + r2 + r3 + r4 + r5 + r6;
```

题目要求统计最高位一侧连续 1 的数量。先对 x 取反, 问题就转化为统计 ~x 的前导 0 数量。程序依次检查最高 16、8、4、2、1 位是否全为 0; 若某一组全为 0, 就把计数增加相应位数, 并将该组左移移出, 再检查剩余部分。

最后再判断一次最高位, 使输入为 -1 时能够得到 32。此时 ~x 为 0, 各段计数之和为 16+8+4+2+1+1=32。

反馈/收获/感悟/总结

参考的重要资料